

Drzewa BST (binary search tree) drzewa poszukiwań binarnych

jako realizacja słownika:

- `search(x, T)`,
- `insert(x, T)`,
- `delete(x, T)`,

gdzie T jest drzewem, a x kluczem.

Zakładamy, że wszystkie klucze są różne.

Def.

Drzewo BST to drzewo binarne, w którym dla dowolnych węzłów x, y zachodzi:

- x jest w lewym poddrzewie o korzeniu $y \Rightarrow x < y$,
- x jest w prawym poddrzewie o korzeniu $y \Rightarrow x > y$.

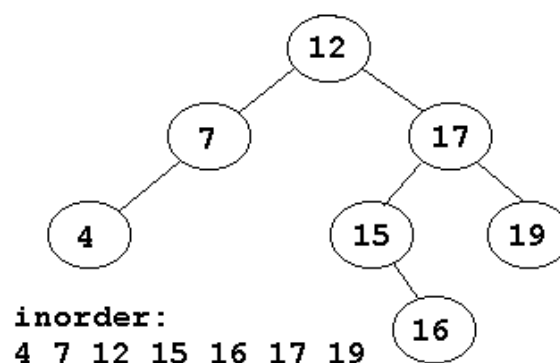
Fakt.

Drzewo binarne jest BST wtedy i tylko wtedy gdy lista jego węzłów w porządku inorder jest ciągiem rosnącym.
słabo rosnącym, jeśli klucze mogą się powtarzać

Dowód.

Z definicji porządku inorder:

lewe (mniejsze) ... korzeń ... prawe (większe)



Operacja wyszukiwania – Search(x, T)

we:

- root = wskaźnik do korzenia drzewa BST T,
- element x,

wy:

- wskaźnik p do węzła zawierającego x,
- NULL jeśli elementu x nie ma w drzewie.

```
p := root;
while p ≠ NULL and x ≠ info(p) do
    if x < info(p) then
        p := left(p);
    else
        p := right(p);
return p;
```

Złożoność, n = liczba węzłów, h(T) = wysokość drzewa:

- operacje dominujące: porównania pól info w węzłach,
- pesymistycznie liczba porównań = h(T)+1,
- pesymistycznie h(T) = n-1,
- zatem pesymistyczna złożoność: $\Theta(n)$.
(złożoność średnia jest znacznie lepsza).

Operacja wstawiania – Insert(x, T)

n węzłów drzewa BST w kolejności rosnącej (inorder):

$$a_1 < a_2 < a_3 < \dots < a_n$$

dzieli przestrzeń kluczy na n+1 przedziałów:

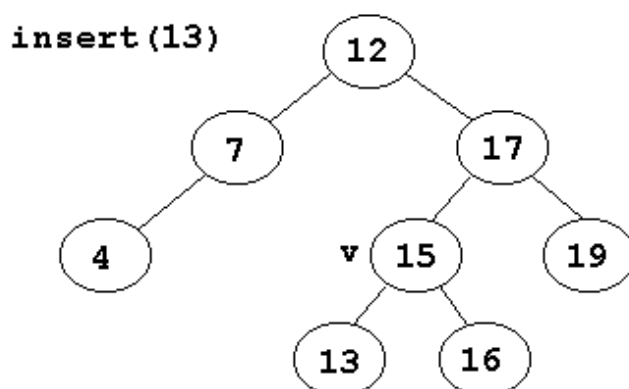
- przedział 0: klucze mniejsze od a_1 ,
- przedział 1: większe od a_1 , mniejsze od a_2 ,
- przedział n-1: większe od a_{n-1} , mniejsze od a_n ,
- przedział n: elementy większe od a_n .

Zatem:

- każdy przedział odpowiada jednemu wskaźnikowi NULL w jakimś węźle drzewa BST;
- na tym wskaźniku kończy się poszukiwanie elementu nieistniejącego w drzewie,
bo wynik wyszukiwania zależy tylko od wyników porównań, a te ostatnie są takie same dla dwóch kluczy z tego samego przedziału;
przy okazji zatem udowodniliśmy, inną metodą, znany fakt, że liczba wskaźników NULL w drzewie binarnym n -wierzchołkowym wynosi $n+1$;
- więc element nowy należy zapisać w nowym liściu dowiązanym w miejscu pustego poddrzewa, na którym zakończy się wyszukiwanie tego elementu;
- technicznie, podczas wyszukiwania należy pamiętać wskaźnik do poprzednika aktualnego węzła na ścieżce.

Na poprzednim rysunku pole $\text{left}(v)$, równe NULL, reprezentuje wartości większe niż 12 i mniejsze niż 15.

Po wstawieniu 13:



Złożoność: jak dla operacji search.

```

Insert (x, T)                                // założenie:  $x \notin T$ 
    s := new node;                            // utwórz nowy węzeł
    info(s) := x;
    left(s) := right(s) := NULL;
    if root(T) = NULL then                    // drzewo jest puste
        root(T) := s;
        return;
    p := root(T);
    prev := NULL; // bieżący i poprzednik na ścieżce
    while p  $\neq$  NULL do
        prev := p;
        if x < info(p) then
            p := left(p);
        else p := right(p);
                                                // dowiąż nowy do prev
    if x < info(prev) then
        left(prev) := s;
    else right(prev) := s;
end.

```

Wersja rekurencyjna operacji Insert(x):

```

InsertR (link &p, Item x)
    // istotne jest przekazanie pierwszego parametru
    // przez referencję
    if p = NULL then
        p := new node;
        info(p) := x;
        left(p) := right(p) := NULL;
        return;
    if x < info(p) then
        InsertR (left(p), x);
    else
        InsertR (right(p), x);
end.

```

Wywołanie funkcji:

```
InsertR(root(T), x).
```

Złożoność: jak wersji iteracyjnej.

Pamięć: $O(n)$ - stos rekursji.

Operacja Delete(x, T)

Usuń z drzewa węzeł zawierający x.

1. Wyszukaj węzeł p taki, że $\text{info}(p)=x$;

```
    Załóżmy: p=left(q)          // q - poprzednik p
```

```
           // jeśli p=right(q) to symetrycznie
```

2. Trzy przypadki:

2.1. $\text{left}(p)=\text{right}(p)=\text{NULL}$: // liść

```
    left(q) := NULL;
```

2.2. $\text{left}(p) \neq \text{NULL}$, $\text{right}(p)=\text{NULL}$: // jeden następnik

```
    left(q) := left(p);
```

```
           // przypadek symetryczny analogicznie
```

2.3. $\text{left}(p) \neq \text{NULL}$, $\text{right}(p) \neq \text{NULL}$: //dwa następniki

```
    s := p$;          // następny w inorder po p
```

```
    // od p idź raz w prawo i do końca w lewo
```

```
    // zachodzi: left(s)=NULL
```

```
    info(p) := info(s); // wartość x skasowana
```

```
    usuń węzeł s stosując 2.1 lub 2.2;
```

end.

```
// symetryczny wariant algorytmu: punkt 2.3:
```

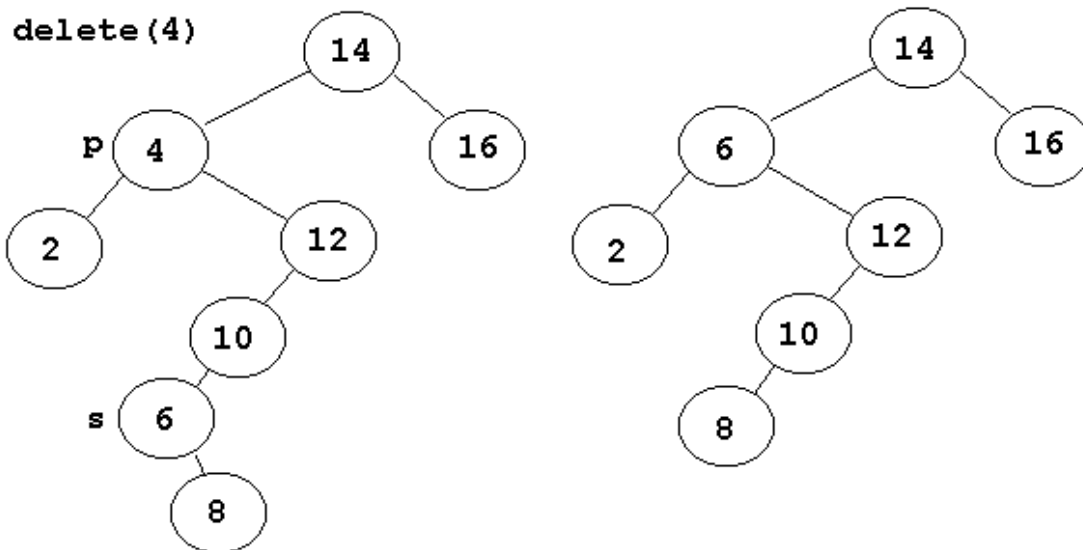
```
    s := $p;          // poprzednik w inorder dla p
```

```
    // od p idź raz w lewo i do końca w prawo
```

```
    // zachodzi: right(s)=NULL
```

```
    info(p) := info(s); // wartość x skasowana
```

```
    usuń węzeł s stosując 2.1 lub 2.2;
```



Uzasadnienie poprawności przypadku 2.3:

w przedziale $(info(p), info(s))$ nie ma żadnego klucza w słowniku realizowanym przez drzewo T.

Złożoność: proporcjonalna do długości ścieżki od korzenia do węzłów p oraz s – pesymistycznie $\Theta(n)$.

Uwagi:

- w przypadku 2.3 usuwamy z drzewa węzeł s - inny niż ten który oryginalnie zawierał klucz x;
- powyższa wersja operacji Delete() jest uważana za standardową,
- nie wydłuża ona żadnej ścieżki od korzenia do liścia, a zatem nie zwiększa wysokości drzewa.

Ćwiczenie:

opracuj algorytm usuwania klucza z drzewa BST wraz z węzłem który go (oryginalnie) zawiera; taki algorytm może zwiększać wysokość drzewa.

Podsumowanie

Jeśli by pod uwagę brać tylko złożoność pesymistyczną, to posługiwanie się strukturą BST byłoby nieopłacalne (dla zwykłej tablicy/listy osiągamy tę samą złożoność).

Twierdzenie

Jeśli do początkowo pustego drzewa BST wstawiamy n parami różnych elementów w losowej kolejności (model permutacyjny), to oczekiwana wysokość drzewa wynosi $\Theta(\log n)$.

Z badań empirycznych:

- losowe wykonywanie operacji Insert i Delete zwiększa średni czas operacji do $\Theta(\sqrt{n})$,
- jeśli w przypadku 2.3 losujemy czy wykorzystywać następny element w porządku inorder czy poprzedni, to oczekiwana długość ścieżki dostępu wynosi $\Theta(\log n)$.

Modyfikacja operacji usuwania:

LazyDelete(x):

- wyszukaj węzeł p zawierający klucz x ;
- oznakuj węzeł p jako usunięty, ale nie odłączaj go od drzewa.

Fakt

W realizacji słownika za pomocą drzewa BST, jeśli

- elementy wstawiane są w losowej kolejności,
 - usuwanie jest w wersji LazyDelete,
 - liczba elementów w drzewie nigdy nie przekracza n ,
- to średnia złożoność pojedynczej operacji słownika wynosi $O(\log n)$.

Wniosek praktyczny:

Jeśli dane dla operacji na słowniku nie są "złośliwie" ułożone, to drzewo BST stanowi prosty i efektywny sposób realizacji tych operacji.